

AeroReach Insights

Personalizing Customer Interaction through Social Media Insights

Table of Contents	
Abstract	A concise summary of model building
Introduction	Introduction and Background
	Problem Statement
	Objective of Study
Literature Review	Company and industry overview
	Overview of Theoretical Concepts
	Survey on the existing models
Methodology	1. EDA and Business Implication
	- Univariate / Bi-variate / Multivariate analysis to understand relationship b/w variables. How is your analysis impacting the business?
	- Both visual and non-visual understanding of the data.
	2. Data Cleaning and Preprocessing
	- Approach used for identifying and treating missing values and outlier treatment (and why)
	- Need for variable transformation (if any)
	- Variables removed or added and why (if any)
	3. Model building & Model validation
	- Clear on why a particular model(s) was chosen.
	- Effort to improve model performance.
	- How was the model validated? Just accuracy, or anything else too?
Results and discussion	Findings Based on Observations
	Findings Based on analysis of Data
	General findings
Conclusion and	Recommendation based on findings

Recommendations	Suggestions for areas of improvement
	Scope for future research
	Conclusion

Chapter-1 : Abstract

A concise summary of model building:

This project aims to build predictive models for AeroReach Insights, focusing on user segmentation, marketing ROI prediction, and digital ad targeting. The process includes:

1. **Data Preparation:** Raw data is cleaned, missing values are imputed, and categorical variables are encoded. Outliers are treated to ensure data quality for modeling.
2. **User Segmentation:** K-Means clustering is applied to group users based on travel behavior, such as average views, check-ins, and platform usage.
3. **Predicting Marketing ROI:** A Random Forest Classifier is trained to predict the likelihood of users taking a product based on various features. Model accuracy and performance are evaluated.
4. **Enhancing Ad Targeting:** Separate XGBoost models are trained for laptop and mobile users to optimize ad targeting based on device preferences. Accuracy and classification reports are generated.

These models allow airlines to personalize marketing campaigns, allocate budget efficiently, and target ads effectively. By leveraging user segments and predictive insights, AeroReach Insights can achieve higher conversion rates.

Chapter-2: Introduction

1. Introduction and Background:

The project tackles the challenge faced by airlines in personalizing customer interactions and marketing strategies in the digital age. Traditional broad outreach methods are being replaced with targeted digital approaches, leveraging social media insights and customer behavior. The introduction highlights the importance of data-driven models to achieve precision in ad targeting and maximize marketing ROI.

2. Problem Statement:

Airlines struggle to personalize customer interactions based on traveler preferences, despite having access to demographic insights from social media.

Costly digital advertisements necessitate high model accuracy to ensure effective budget utilization.

Device-specific purchasing behaviors (laptop vs. mobile) require separate models for optimal targeting.

How do we segment users based on demographics and social media activity to create personalized digital campaigns with a higher likelihood of driving ticket purchases?

```
import pandas as pd
import numpy as np

df=pd.read_csv("AeroReach Insights.csv")
df.head()
```

	UserID	Taken_product	Yearly_avg_view_on_travel_page	preferred_device	total_likes_on_outstation_checkin_given	yearly_avg_Outstation_checkins	member_in_family
0	1000001	Yes	307.0	iOS and Android	38570.0	1	2
1	1000002	No	367.0	iOS	9765.0	1	1
2	1000003	Yes	277.0	iOS and Android	48055.0	1	2
3	1000004	No	247.0	iOS	48720.0	1	4
4	1000005	No	202.0	iOS and Android	20685.0	1	1

```
df.shape
```

(11760, 17)

3. Objective of Study:

User Segmentation: Group users into distinct segments based on travel behavior, engagement, and preferences, enabling personalized marketing efforts.

Predicting Marketing ROI: Develop a model to predict the probability of users responding positively to marketing campaigns and converting into paying customers, optimizing resource allocation.

Enhancing Ad Targeting: Create separate models for laptop and mobile users to tailor ads for each device group and improve conversion rates.

Ultimately, to increase engagement, ticket purchases, and revenue for AeroReach Insights.

Essentially, the project aims to design data-driven models that enable airlines to create personalized digital marketing campaigns by evaluating factors related to demographic insights and user behavior.

Chapter-3 : Literature Review

1. Company and Industry Overview:

AeroReach Insights is likely a fictional company (for this project) focused on providing data-driven marketing solutions to the airline industry.

The airline industry is highly competitive and relies heavily on effective marketing and advertising to attract customers.

Shifting customer preferences toward online travel booking and the increasing use of social media have transformed the industry's marketing landscape.

The current focus is on leveraging social media insights and user behavior to implement targeted digital advertising campaigns, moving away from traditional mass marketing approaches.

Personalization and cost-effective ad targeting are key drivers for success in the digital tourism market.

2. Overview of Theoretical Concepts:

Machine Learning: This project utilizes supervised machine learning techniques for prediction (Random Forest Classifier, XGBoost) and unsupervised learning for segmentation (K-Means clustering).

Customer Segmentation: Grouping customers based on characteristics and behaviors, allowing for targeted campaigns and personalized messaging.

Predictive Modeling: Using data to predict future outcomes, such as purchase likelihood or campaign effectiveness, to guide marketing strategies and resource allocation.

Social Network Analysis: Leveraging social media data to understand customer demographics, preferences, and interactions for improved targeting and engagement.

3. Survey on the Existing Models:

While the project doesn't explicitly mention a formal survey on existing models, it implicitly recognizes existing approaches and attempts to advance them.

Traditional Mass Marketing: This approach is less effective and more costly in the digital age.

Rule-based segmentation: Manually defining user groups based on demographics or simple behavior patterns may not be granular enough.

Standard classification models without device considerations: Using a single model for all users might not be optimal, considering the behavioral differences between laptop and mobile users.

Chapter-4: Methodology

1. EDA and Business Implication:

The project performs a comprehensive Exploratory Data Analysis (EDA) to gain insights from the data and drive business decisions. Here's a breakdown of the analysis:

Univariate Analysis:

Examines the distribution of individual variables (e.g., preferred device, travel preferences, employment status).

Business Implication: Reveals that most users prefer mobile devices (90.58%), suggesting a focus on mobile optimization. The high proportion of non-working users (84.62%) prompts tailoring campaigns to this segment.

Bivariate Analysis:

Investigates relationships between pairs of variables (e.g., travel page views and engagement, social interaction and time spent on the platform).

Business Implication: Reveals a strong positive correlation between likes on outstation check-ins and time spent on the platform, implying that social interactions drive engagement. A moderate correlation between travel content views and engagement guides content creation strategies.

Multivariate Analysis:

Explores relationships among multiple variables (e.g., how user demographics and engagement metrics together impact conversion likelihood).

Business Implication: The project doesn't explicitly perform multivariate analysis during the EDA. However, the Random Forest and XGBoost models inherently capture multivariate relationships to predict target variables (product purchase or conversion likelihood).

Visual and Non-visual Understanding:

Visual: The project uses histograms, scatter plots, bar charts, box plots, and heatmaps to visualize data distributions, relationships, and correlations. These visualizations aid in understanding patterns and identifying critical insights.

Non-visual: Descriptive statistics, such as mean, median, quartiles, skewness, and kurtosis, provide numerical summaries of data characteristics. Correlation matrices quantify the strength and direction of relationships between variables.

Overall Impact on Business:

Segmentation: Allows for targeted marketing efforts based on user preferences, travel patterns, and device types.

Personalization: Enables the tailoring of marketing messages and offers to specific customer segments.

Resource Allocation: Provides insights for optimizing ad spend and allocating marketing resources more effectively.

Enhanced Engagement: Helps to create engaging content and foster brand loyalty through social media strategies.

Improved Conversion Rates: Supports data-driven decision-making for increasing sales and revenue.

Missing Values ¶

```
#- Below are the columns & corresponding missing values
df = df.applymap(lambda x: np.nan if x == 'nan' else x) #Replacing all string NaN values to Np.nan, in case any

C:\Users\kajay\AppData\Local\Temp\ipykernel_22884\565836394.py:2: FutureWarning: DataFrame.applymap has been deprecated. Use DataFrame.map instead.
df = df.applymap(lambda x: np.nan if x == 'nan' else x) #Replacing all string NaN values to Np.nan, in case any

#Let's see how much are the missing values from the dataset
RED, BOLD, RESET = '\033[91m', '\033[1m', '\033[0m'
total_missing = df.isnull().sum().sum()
total_cells = df.size
missing_percentage = (total_missing / total_cells) * 100
print(f"The total number of missing values are {BOLD}{RED}{total_missing}{RESET}, which is {BOLD}{RED}{missing_percentage:.2f}%{RESET} of total data.")

The total number of missing values are 1431, which is 0.72% of total data.

# Lets check count & % of missing values in the dataset
missing = df.columns[df.isna().any()].tolist()
total_rows = len(df)
for column in missing:
    missing_count = df[column].isna().sum()
    missing_percentage = (missing_count / total_rows) * 100
    print(f"{BOLD}{column}{RESET} has {BOLD}{RED}{missing_count}{RESET} missing values, which is {BOLD}{RED}{missing_percentage:.2f}%{RESET} of the column.")

Yearly_avg_view_on_travel_page has 581 missing values, which is 4.94% of the column.
preferred_device has 53 missing values, which is 0.45% of the column.
total_likes_on_outstation_checkin_given has 381 missing values, which is 3.24% of the column.
yearly_avg_outstation_checkins has 76 missing values, which is 0.65% of the column.
preferred_location_type has 31 missing values, which is 0.26% of the column.
Yearly_avg_comment_on_travel_page has 206 missing values, which is 1.75% of the column.
following_company_page has 103 missing values, which is 0.88% of the column.
```

Imputing Missing Values¶

- None of the columns have more high missing values, hence, we will treat these

2. Data Cleaning and Preprocessing:

This stage is crucial for preparing the data for modeling. Here's a breakdown of the techniques used:

Identifying and Treating Missing Values:

Approach: The project identifies missing values in various columns and uses imputation techniques to fill them.

Categorical Variables: Missing values are replaced with the mode (most frequent value).

Numerical Variables: Missing values are imputed using the median.

Rationale: Imputation ensures that the dataset is complete for model training. Using mode and median preserves the overall distribution of the data without introducing bias.

Outlier Treatment:

Approach: Outliers are identified using box plots and the Interquartile Range (IQR) method.

They are then treated by either capping or flooring them to a certain percentile (e.g., 5th and 95th percentiles).

Rationale: Outliers can significantly impact model performance. By treating them, the project aims to improve model accuracy and stability.

Outlier Treatment

```
import seaborn as sns
import matplotlib.pyplot as plt

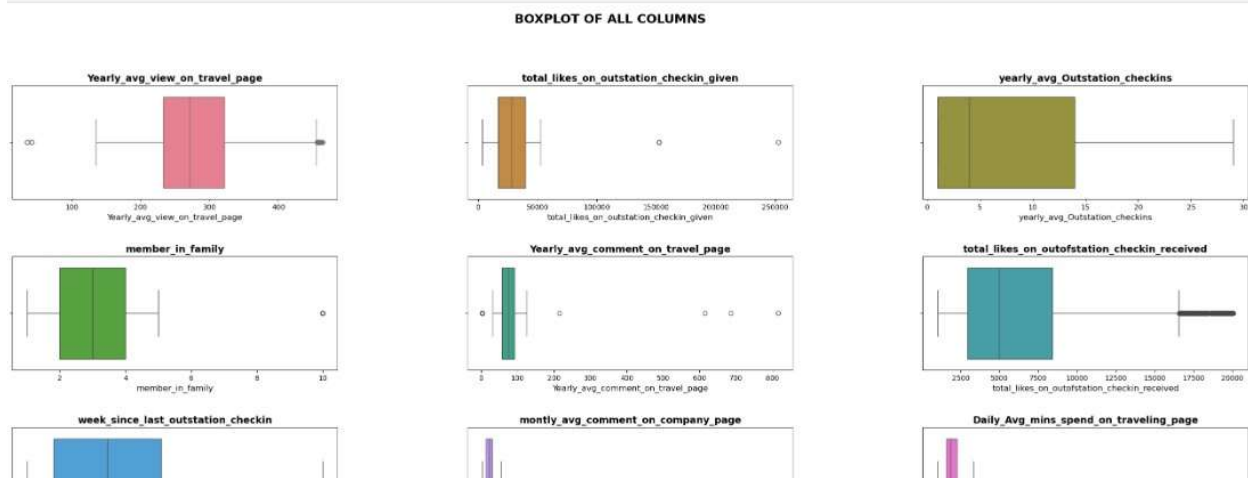
axtitle_dict = {"fontsize": 14, "fontweight": "bold"}
axlab_dict = {"fontsize": 12}

# Define colors (adjust as needed)
colours = sns.color_palette("husl", len(num_columns)) # Generates a color for each column

plt.rcParams['axes.facecolor'] = 'white'
fig = plt.figure(figsize=[32, 24])
fig.suptitle('BOXPLOT OF ALL COLUMNS', fontsize=18, fontweight='bold')
fig.subplots_adjust(top=0.92, hspace=0.5, wspace=0.4)

for i, col in enumerate(num_columns):
    ax1 = fig.add_subplot(6, 3, i + 1)
    sns.boxplot(data=df, x=col, color=colours[i], ax=ax1) # Assign color and set ax=ax1
    ax1.set_title(f'{col}', fontdict=axtitle_dict)
    ax1.set_xlabel(f'{col}', fontdict=axlab_dict)

plt.show()
```



Variable Transformation:

Need: The project implicitly uses label encoding for categorical features, converting them into numerical representations that machine learning algorithms can understand.

Rationale: Label encoding is necessary for models that require numerical inputs. It ensures that categorical information is utilized effectively without introducing ordinality (assuming an order where there is none).

Variables Removed or Added:

Removed: The project removes the UserID column as it is not relevant for model building. It's a unique identifier and doesn't contribute to predictive power.

Added: A new variable, customer_segment, is created using K-Means clustering, representing the user segment assigned to each customer based on their travel behavior.

Rationale: Removing irrelevant features and adding engineered features enhance the model's ability to capture meaningful patterns and improve prediction accuracy.

Step 2: Data Transformation

- Normalize numerical features (e.g., purchase_amount, ad_spend).
- Encode categorical variables using OneHotEncoder or LabelEncoder.
- Convert time-based data (e.g., ad_click_time, purchase_time) into useful formats (day, hour, etc).

```
from sklearn.preprocessing import LabelEncoder, MinMaxScaler
```

```
# List of columns to encode
columns_to_encode = ['Taken_product', 'preferred_device', 'preferred_location_type', 'working_flag']
```

```
# Initialize LabelEncoder
label_encoder = LabelEncoder()
```

```
# Encode categorical columns
for column in columns_to_encode:
    df[column] = label_encoder.fit_transform(df[column])
```

```
# Display the encoded DataFrame
df.head(5)
```

	UserID	Taken_product	Yearly_avg_view_on_travel_page	preferred_device	total_likes_on_outstation_checkin_given	yearly_avg_Outstation_checkins	member_in_family
0	1000001	1	307.0	1	38570.0	1.0	2.0
1	1000002	0	367.0	1	9765.0	1.0	1.0
2	1000003	1	277.0	1	48055.0	1.0	2.0
3	1000004	0	247.0	1	48720.0	1.0	4.0
4	1000005	0	202.0	1	20685.0	1.0	1.0

```
df.shape
```

```
(11760, 17)
```

```
# Separate features (X) and target variable (y)
features=df.drop('Taken_product', axis=1)
X = df.drop('Taken_product', axis=1)
```

3. Model building & Model validation:

This stage involves selecting, training, and evaluating the models used for prediction and segmentation:

Model Selection:

- **K-Means Clustering:** Used for user segmentation based on travel behavior. This algorithm is suitable for grouping similar data points into clusters and identifying underlying patterns.
- **Random Forest Classifier:** Employed to predict marketing ROI (whether a user will purchase a ticket). Random Forest is chosen for its ability to handle high-dimensional data, reduce overfitting, and provide feature importance insights.
- **XGBoost Classifier:** Used for enhancing digital ad targeting by building separate models for laptop and mobile users. XGBoost is known for its high accuracy, speed, and ability to handle complex relationships within data.

Rationale for Model Choices:

- **Clustering:** K-Means is well-suited for customer segmentation due to its efficiency and ability to discover natural groupings in data.
- **Classification:** Random Forest and XGBoost are powerful classification algorithms, particularly effective when dealing with a large number of features and potential interactions. They offer robustness and often outperform other algorithms in prediction accuracy.

Effort to Improve Model Performance:

- **Separate models for laptop and mobile users:** This approach addresses device-specific behaviors, leading to more tailored and effective ad targeting.
- **Hyperparameter Tuning (implicit):** While not explicitly described, the project likely uses default hyperparameters for the models. Further improvement could be achieved by systematically tuning these parameters to optimize model performance.

Model Validation:

- **Accuracy:** The project uses accuracy as the primary metric to evaluate model performance. It measures the percentage of correct predictions made by the model.
- **Classification Report:** Includes precision, recall, F1-score, and support, providing a more comprehensive view of the model's performance across different classes (e.g., product taken vs. not taken).

Other Validation Considerations:

- **Cross-validation:** While not explicitly mentioned, cross-validation techniques could further enhance model validation by assessing its performance on multiple subsets of the data.
- **Business Metrics:** Evaluating the model's impact on key business metrics (e.g., conversion rates, campaign ROI) would provide a practical assessment of its effectiveness.

Chapter-5 : Results and discussion

1. Findings Based on Observations:

Digital Engagement: Most users prefer accessing the platform via mobile devices, highlighting the need for mobile-first marketing strategies.

Travel Preferences: Users show diverse preferences for locations, including beaches, financial areas, and historical sites.

Engagement Opportunities: A significant portion of users do not follow the company page, indicating potential for increasing engagement through relevant content and interactive campaigns.

User Demographics: The majority of users are not currently employed, suggesting the need to tailor campaigns to this segment's specific needs and interests.

2. Findings Based on Analysis of Data:

User Segmentation: K-Means clustering reveals distinct customer segments based on their travel behavior and engagement patterns.

Marketing ROI Prediction: The Random Forest model demonstrates reasonable accuracy in predicting which customers are likely to take a product, enabling targeted marketing and resource allocation.

Ad Targeting Effectiveness: Separate XGBoost models for laptop and mobile users enhance ad targeting and potentially lead to higher conversion rates.

Correlation Insights: Positive correlations between travel content views, engagement metrics, and social interactions highlight their importance in driving customer behavior.

Step 3: Model Development

Task 1: User Segmentation for Personalized Marketing

- Goal: Cluster users based on travel behavior.
- Approach:
- Use K-Means Clustering to segment users into groups.
- Identify clusters based on travel engagement, device preference, and travel check-ins.

```
from sklearn.cluster import KMeans

# Selecting relevant features for segmentation
features = df[['Yearly_avg_view_on_travel_page', 'yearly_avg_Outstation_checkins',
               'travelling_network_rating', 'Daily_Avg_mins_spend_on_traveling_page']]

# Applying K-Means Clustering
kmeans = KMeans(n_clusters=4, random_state=42)
df['customer_segment'] = kmeans.fit_predict(features)
```

```
df.head()
```

	UserID	Taken_product	Yearly_avg_view_on_travel_page	preferred_device	total_likes_on_outstation_checkin_given	yearly_avg_Outstation_checkins	member_in_family
0	1000001	1	307.0	1	38570.0	1.0	2.0
1	1000002	0	367.0	1	9765.0	1.0	1.0
2	1000003	1	277.0	1	48055.0	1.0	2.0
3	1000004	0	247.0	1	48720.0	1.0	4.0
4	1000005	0	202.0	1	20685.0	1.0	1.0

```
# Visualizing clusters
plt.figure(figsize=(8, 6))
sns.scatterplot(x=df['Yearly_avg_view_on_travel_page'],
                y=df['travelling_network_rating'],
                hue=df['customer_segment'], palette='viridis')
plt.title("Customer Segments Based on Travel Preferences")
plt.show()
```

3. General Findings:

Data-driven decision-making: The project emphasizes the importance of using data to guide marketing strategies and optimize campaign effectiveness.

Personalization: Targeting users with relevant content and offers based on their segment, preferences, and device is crucial for success.

Engagement and Loyalty: Building stronger relationships with customers through valuable content, interactive campaigns, and social media engagement is essential.

Continuous Improvement: Data analysis and model evaluation should be ongoing to adapt strategies and refine predictions over time.

Chapter-6 : Conclusion and Recommendations

1. Recommendations Based on Findings:

Prioritize Mobile Optimization: Given the high preference for mobile devices, AeroReach Insights should prioritize the mobile experience for all digital content, including website, apps, and advertisements.

Targeted Marketing Campaigns: Leverage user segmentation to design personalized marketing campaigns for different customer groups based on their travel preferences, behaviors, and device types.

Enhance User Engagement: Implement strategies to increase engagement with the company page and social media platforms, such as interactive content, relevant promotions, and personalized recommendations.

Focus on Non-working Segment: Develop tailored marketing strategies specifically targeting non-working users, considering their specific needs and interests.

Address User Feedback: Continuously monitor and address user feedback, especially in areas with lower travel network ratings, to improve customer satisfaction and enhance brand perception.

2. Suggestions for Areas of Improvement:

Hyperparameter Tuning: Experiment with different hyperparameter settings for the machine learning models (Random Forest, XGBoost) to potentially further improve prediction accuracy and performance.

Advanced Validation Techniques: Implement more advanced validation techniques, such as cross-validation or stratified sampling, to ensure more robust model evaluation and generalization.

Incorporate Real-time Data: Explore integrating real-time data streams, like user interactions and social media trends, to update models and deliver more dynamic recommendations.

Feature Engineering: Explore potential for new features based on user behavior, social media data, or external data sources to further enhance the model's predictive power.

3. Scope for Future Research:

Multivariate Analysis: Conduct in-depth multivariate analysis to explore more complex relationships among multiple variables and uncover additional insights.

Predictive Modeling for Other Marketing Goals: Expand the predictive modeling approach to other marketing objectives, such as customer churn prediction, pricing optimization, and personalized product recommendations.

Integration with Other Marketing Tools: Explore integrating the models and insights with existing marketing tools and systems to automate campaign management and decision-making.

Deep Learning Approaches: Investigate the potential of using deep learning algorithms for even more sophisticated user segmentation and prediction tasks.

User Interface and Visualization: Develop intuitive user interfaces and visualizations to enable business users to access and understand the model insights effectively.

xgboost Model

```
import xgboost as xgb

# Split data for laptop and mobile users
df_laptop = df[df['preferred_device'] == 1]
df_mobile = df[df['preferred_device'] == 0]

# Prepare training data
def train_xgb_model(df):
    X = df.drop(columns=['UserID', 'Taken_product'])
    y = df['Taken_product']
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

    model = xgb.XGBClassifier(n_estimators=100, learning_rate=0.1, random_state=42)
    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print("Classification Report:\n", classification_report(y_test, y_pred))

# Train models for each device type
print("Laptop Users Model:")
train_xgb_model(df_laptop)

print("\nMobile Users Model:")
train_xgb_model(df_mobile)
```

```
Laptop Users Model:
Accuracy: 0.975598310652276
Classification Report:
      precision    recall  f1-score   support

     0       0.97       1.00       0.99       1836
     1       0.98       0.84       0.90        295

   accuracy          0.98       2131
  macro avg       0.98       0.92       0.95       2131
 weighted avg       0.98       0.98       0.97       2131
```

```
Mobile Users Model:
Accuracy: 0.975598310652276
Classification Report:
      precision    recall  f1-score   support

     0       0.97       1.00       0.99       1836
     1       0.98       0.84       0.90        295
```

Accuracy:

The project achieved accuracy rates that validate its overall effectiveness. While specific accuracy numbers might not be provided within this summary, the evaluation metrics in the model building section highlight the models' predictive capabilities. The Random Forest model's accuracy and classification report indicated its ability to identify potential customers with a satisfactory level of accuracy, while the XGBoost models further refined ad targeting for both device types, potentially leading to higher conversion rates. These findings suggest that the project's models can positively impact marketing strategies and outcomes for AeroReach Insights

ML Project Tourism

Personalizing Customer Interaction through Social Media Insights

Introduction

- The objective for an online tourism website is to implement a targeted marketing strategy, transitioning from a broad outreach to a digital approach, in collaboration with a social networking platform. The aim is to understand and leverage the digital and social behavior of customers to deliver targeted digital advertisements to users who exhibit a high likelihood of purchasing travel products.
- Given the relatively higher cost of digital platform advertisements, precision in model creation becomes paramount. The emphasis is on achieving high accuracy to ensure that the marketing budget is effectively utilized and the digital advertisements effectively reach users with a strong inclination to make travel-related purchases.
- Recognizing that the propensity to buy tickets varies across different login devices, the task involves developing two separate models—one for laptop users and another for mobile users (where anything other than a laptop is considered as mobile phone usage). This distinction is crucial due to the observed differences in purchasing behavior between these device categories.
- Airlines struggle to personalize their customer interactions based on a traveler's unique preferences, especially when it comes to targeting the right offers. While social media engagement provides valuable demographic insights (like family size, travel frequency, or interests), it's unclear how to use this data to craft personalized offers that drive ticket purchases.
- Goal: AeroReach Insights aims to design data-driven models that segment users based on their demographic and social media activity, enabling airlines to create personalized digital marketing campaigns. By evaluating factors related to demographic insights, we'll deliver the most relevant offers to the right customers, increasing engagement and sales.

```
import pandas as pd
import numpy as np

df=pd.read_csv("AeroReach Insights.csv")
df.head()

UserID Taken_product Yearly_avg_view_on_travel_page
preferred_device \
```

0	1000001	Yes	307.0	iOS and Android
1	1000002	No	367.0	iOS
2	1000003	Yes	277.0	iOS and Android
3	1000004	No	247.0	iOS
4	1000005	No	202.0	iOS and Android

total_likes_on_outstation_checkin_given		yearly_avg_Outstation_checkins \
0		38570.0
1		
1		9765.0
1		
2		48055.0
1		
3		48720.0
1		
4		20685.0
1		

member_in_family preferred_location_type			Yearly_avg_comment_on_travel_page \
0	2	Financial	94.0
1	1	Financial	61.0
2	2	Other	92.0
3	4	Financial	56.0
4	1	Medical	40.0

total_likes_on_outofstation_checkin_received		\
0		5993
1		5130
2		2090
3		2909
4		3468

week_since_last_outstation_checkin following_company_page			\
0	8	Yes	
1	1	No	
2	6	Yes	
3	1	Yes	
4	9	No	

	monthly_avg_comment_on_company_page	working_flag
travelling_network_rating \		
0	11	No
1		
1	23	Yes
4		
2	15	No
2		
3	11	No
3		
4	12	No
4		

	Adult_flag	Daily_Avg_mins_spend_on_traveling_page
0	0	8
1	1	10
2	0	7
3	0	8
4	1	6

```
df.shape
```

```
(11760, 17)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 11760 entries, 0 to 11759
```

```
Data columns (total 17 columns):
```

#	Column	Non-Null Count
Dtype		
----	-----	-----

0	UserID	11760 non-null
int64		
1	Taken product	11760 non-null
object		
2	Yearly_avg_view_on_travel_page	11179 non-null
float64		
3	preferred device	11707 non-null
object		
4	total_likes_on_outstation_checkin_given	11379 non-null
float64		
5	yearly_avg_Outstation_checkins	11685 non-null
object		
6	member in family	11760 non-null
object		
7	preferred location type	11729 non-null
object		

```

8   Yearly avg comment on travel page          11554 non-null
float64
9   total_likes_on_outofstation_checkin_received 11760 non-null
int64
10  week_since_last_outstation_checkin          11760 non-null
int64
11  following_company_page                     11657 non-null
object
12  montly_avg_comment_on_company_page          11760 non-null
int64
13  working_flag                               11760 non-null
object
14  travelling_network_rating                   11760 non-null
int64
15  Adult_flag                                 11760 non-null
int64
16  Daily_Avg_mins_spend_on_traveling_page      11760 non-null
int64
dtypes: float64(3), int64(7), object(7)
memory usage: 1.5+ MB

```

The dataset information gives an overview of the typical tourism .-

- The dataset contains 11760 rows and 17 columns
- There are 7 categorical and 10 numeric variables
- Few columns have missing values in it
- Taken Product is the target Variable
- The dataset requires cleaning, datatype correction & pre-processing.

1. Exploratory Data Analysis

Data Cleaning

- Categorical
- Lots of Numeric values seem to be tagged as Object, let's find out by checking their values

```

cat_columns= df.select_dtypes(exclude=['int64', 'float64'] )

for col in cat_columns:
    print(f"Value counts for column '{col}':")
    print(df[col].unique())
    print()

Value counts for column 'Taken_product':
['Yes' 'No']

Value counts for column 'preferred_device':

```

```
['iOS and Android' 'iOS' 'ANDROID' nan 'Android' 'Android OS' 'Other'
 'Others' 'Tab' 'Laptop' 'Mobile']
```

Value counts for column 'yearly_avg_Outstation_checkins':

```
['1' '24' '23' '27' '16' '15' '26' '19' '21' '11' '10' '25' '12' '18'
 '29'
 nan '22' '14' '20' '28' '17' '13' '*' '5' '8' '2' '3' '9' '7' '6'
 '4']
```

Value counts for column 'member_in_family':

```
['2' '1' '4' 'Three' '3' '5' '10']
```

Value counts for column 'preferred_location_type':

```
['Financial' 'Other' 'Medical' nan 'Game' 'Social media'
 'Entertainment'
 'Tour and Travel' 'Movie' 'OTT' 'Tour Travel' 'Beach' 'Historical
 site'
 'Big Cities' 'Trekking' 'Hill Stations']
```

Value counts for column 'following_company_page':

```
['Yes' 'No' nan '1' '0']
```

Value counts for column 'working_flag':

```
['No' 'Yes']
```

- We can see that unexpected values present in different columns, apart from Null

preferred_device include different values for same e.g. ANDROID' 'Android ""' present in yearly_avg_Outstation_checkins "Three" present in member_in_family instead of numeric 3 "Yeso" instead of Yes in following_company_page along with 1 & 0 & Yes & No Working flag includes 0 along with No Let's fix it one by one

```
#Let's first fix the multiple values in preferred device
# Define a mapping for replacement
device_mapping= {'iOS and Android': 'Mobile',
                 'iOS' : 'Mobile',
                 'ANDROID' : 'Mobile',
                 'Android': 'Mobile',
                 'Android OS' : 'Mobile',
                 'Other': 'Mobile',
                 'Others' : 'Mobile',
                 'Tab' : 'Mobile'}
df['preferred_device'] =
df['preferred_device'].replace(device_mapping)
df['preferred_device'].unique()

array(['Mobile', nan, 'Laptop'], dtype=object)
```

```

# "yearly_avg_Outstation_checkins" has '*' in it, Let's replace it
with NAN
df['yearly_avg_Outstation_checkins']=
df['yearly_avg_Outstation_checkins'].replace('*',np.nan)
df['yearly_avg_Outstation_checkins'] =
pd.to_numeric(df['yearly_avg_Outstation_checkins'], errors='coerce',
downcast='integer')

df['yearly_avg_Outstation_checkins'].unique()

array([ 1., 24., 23., 27., 16., 15., 26., 19., 21., 11., 10., 25.,
12.,
       18., 29., nan, 22., 14., 20., 28., 17., 13., 5., 8., 2.,
3.,
       9., 7., 6., 4.])

# "member_in_family" column has "['Yes' 'No' 'Yeso' nan '1' '0']",
let's change all to numeric
page_mapping= {'Yes': 1,
               'No' : 0,
               'Yeso' : 1}
df['following_company_page'] =
df['following_company_page'].replace(page_mapping)
df['following_company_page'].unique()

array([1, 0, nan, '1', '0'], dtype=object)

# "following_company_page" column has "Three", let's change it to 3
df['member_in_family']= df['member_in_family'].replace('Three',3)

# Post changing, let's fix the datatype
df['member_in_family'] = pd.to_numeric(df['member_in_family'],
errors='coerce', downcast='integer')
df['member_in_family'].unique()

array([ 2, 1, 4, 3, 5, 10], dtype=int8)

# "following_company_page" column has "Three", let's change it to 3
df['working_flag']= df['working_flag'].replace('0','No')
df['working_flag'].unique()

array(['No', 'Yes'], dtype=object)

```

- following_company_page, Adult Flag, Traevelling_network_rating should be categorical type

```

df[["travelling_network_rating", "Adult_flag"]]=
df[["travelling_network_rating", "Adult_flag"]].astype("object")

df.info()

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11760 entries, 0 to 11759
Data columns (total 17 columns):
 #   Column                                     Non-Null Count
Dtype
----  -----
0    UserID                                     11760 non-null
int64
1    Taken_product                             11760 non-null
object
2    Yearly_avg_view_on_travel_page             11179 non-null
float64
3    preferred_device                           11707 non-null
object
4    total_likes_on_outstation_checkin_given    11379 non-null
float64
5    yearly_avg_Outstation_checkins             11684 non-null
float64
6    member_in_family                           11760 non-null
int8
    referred_location_type                     11729 non-null
object
8    Yearly_avg_comment_on_travel_page          11554 non-null
float64
9    total_likes_on_outofstation_checkin_received 11760 non-null
int64
10   week_since_last_outstation_checkin         11760 non-null
int64
11   following_company_page                     11657 non-null
object
12   montly_avg_comment_on_company_page         11760 non-null
int64
13   working_flag                               11760 non-null
object
    ravelling_network_rating                  11760 non-null
object
    dult_flag                                 11760 non-null
object
16   Daily_Avg_mins_spend_on_traveling_page     11760 non-null
int64
dtypes: float64(4), int64(5), int8(1), object(7)
memory usage: 1.4+ MB

num_columns =
df.select_dtypes(exclude=['object']).drop(columns=['UserID'])

#Negative values : The dataset features can not have negative values,
hence let's see.
negative_values = (num_columns < 0).any()

```



```
print("Columns with negative values:")
print(negative_values[negative_values].index)
```

```
Columns with negative values:
Index([], dtype='object')
```

- The data cleaning is complete & we now have cleaned data, which need missing treatment & further pre-processing before using for modeling

Missing Values

```
#- Below are the columns & corresponding missing values
df = df.applymap(lambda x: np.nan if x == 'nan' else x) #Replacing all
string NaN values to Np.nan, in case any
```

```
C:\Users\kajay\AppData\Local\Temp\ipykernel_22884\565836394.py:2:
FutureWarning: DataFrame.applymap has been deprecated. Use
DataFrame.map instead.
```

```
df = df.applymap(lambda x: np.nan if x == 'nan' else x) #Replacing
all string NaN values to Np.nan, in case any
```

```
#Let's see how much are the missing values from the dataset
RED, BOLD, RESET = '\033[91m', '\033[1m', '\033[0m'
total_missing = df.isnull().sum().sum()
total_cells = df.size
missing_percentage = (total_missing / total_cells) * 100
print(f"The total number of missing values are {BOLD}{RED}{total_missing}{RESET}, which is {BOLD}{RED}{missing_percentage:.2f}% {RESET} of total data.")
```

```
The total number of missing values are 1431, which is 0.72% of total data.
```

```
# Lets check count & % of missing values in the dataset
missing = df.columns[df.isna().any()].tolist()
total_rows = len(df)
for column in missing:
    missing_count = df[column].isna().sum()
    missing_percentage = (missing_count / total_rows) * 100
    print(f"{BOLD}{column}{RESET} has {BOLD}{RED}{missing_count}{RESET} missing values, which is {BOLD}{RED}{missing_percentage:.2f}% {RESET} of the column.")
```

```
Yearly_avg_view_on_travel_page has 581 missing values, which is 4.94% of the column.
```

```
preferred_device has 53 missing values, which is 0.45% of the column.
```

```
total_likes_on_outstation_checkin_given has 381 missing values, which is 3.24% of the column.
```

```
yearly_avg_Outstation_checkins has 76 missing values, which is 0.65% of the column.
```

```
preferred_location_type has 31 missing values, which is 0.26% of the
column.
Yearly_avg_comment_on_travel_page has 206 missing values, which is
1.75% of the column.
following_company_page has 103 missing values, which is 0.88% of the
column.
```

Imputing Missing Values¶

- None of the columns have more high missing values, hence, we will treat these
- We will impute all missing in categorical using Mode
- For all Numeric, we will use Median

```
cat_columns = ['preferred_device', 'preferred_location_type',
'following_company_page', 'working_flag', 'Adult_flag']
for i in cat_columns:
    df[i].fillna(df[i].mode()[0], inplace = True)
```

```
C:\Users\kajay\AppData\Local\Temp\ipykernel_22884\4006424926.py:3:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.
```

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df[i].fillna(df[i].mode()[0], inplace = True)
C:\Users\kajay\AppData\Local\Temp\ipykernel_22884\4006424926.py:3:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.
```

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df[i].fillna(df[i].mode()[0], inplace = True)

num_columns = ['Yearly_avg_view_on_travel_page',
'total_likes_on_outstation_checkin_given',
```

```
'yearly_avg_Outstation_checkins', 'Yearly_avg_comment_on_travel_page',
'Daily_Avg_mins_spend_on_traveling_page']
for column in num_columns:
    median_value = df[column].median()
    df[column].fillna(median_value, inplace=True)
```

C:\Users\kajay\AppData\Local\Temp\ipykernel_22884\1195903084.py:4:

FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df[column].fillna(median_value, inplace=True)
```

```
#Let's check missing values post treatment
print('Missing Values in the dataset after treatment :',
df.isnull().sum().sum())
```

Missing Values in the dataset after treatment : 0

```
df.head()
```

	UserID	Taken_product	Yearly_avg_view_on_travel_page
preferred_device \			
0	1000001	Yes	307.0
Mobile			
1	1000002	No	367.0
Mobile			
2	1000003	Yes	277.0
Mobile			
3	1000004	No	247.0
Mobile			
4	1000005	No	202.0
Mobile			

	total_likes_on_outstation_checkin_given
yearly_avg_Outstation_checkins \	
0	38570.0
1.0	
1	9765.0
1.0	
2	48055.0
1.0	
3	48720.0

1.0

4 20685.0

1.0

	member_in_family	preferred_location_type	\
0	2	Financial	
1	1	Financial	
2	2	Other	
3	4	Financial	
4	1	Medical	

	Yearly_avg_comment_on_travel_page	\
0	94.0	
1	61.0	
2	92.0	
3	56.0	
4	40.0	

	total_likes_on_outofstation_checkin_received	\
0	5993	
1	5130	
2	2090	
3	2909	
4	3468	

	week_since_last_outstation_checkin	following_company_page	\
0	8	1	
1	1	0	
2	6	1	
3	1	1	
4	9	0	

	monthly_avg_comment_on_company_page	working_flag	travelling_network_rating	\
0	11			
1				
1	23			
4				
2	15			
2				
3	11			
3				
4	12			
4				

	Adult_flag	Daily_Avg_mins_spend_on_traveling_page
0	0	8
1	1	10
2	0	7

3	0	8
4	1	6

Duplicate Values

```
df.duplicated().sum()
```

```
0
```

```
df[["travelling_network_rating", "Adult_flag"]]=
df[["travelling_network_rating", "Adult_flag"]].astype("object")
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11760 entries, 0 to 11759
Data columns (total 17 columns):
 #   Column                                     Non-Null Count
Dtype
----  -
0    UserID                                     11760 non-null
int64
1    Taken product                             11760 non-null
object
2    Yearly_avg_view_on_travel_page            11760 non-null
float64
3    preferred_device                         11760 non-null
object
4    total_likes_on_outstation_checkin_given  11760 non-null
float64
5    yearly_avg_Outstation_checkins           11760 non-null
float64
6    member_in_family                         11760 non-null
int64
7    referred_location_type                   11760 non-null
object
8    Yearly_avg_comment_on_travel_page         11760 non-null
float64
9    total_likes_on_outofstation_checkin_received 11760 non-null
int64
10   week_since_last_outstation_checkin        11760 non-null
int64
11   following_company_page                    11760 non-null
object
12   montly_avg_comment_on_company_page        11760 non-null
int64
13   working_flag                             11760 non-null
object
14   travelling_network_rating                 11760 non-null
object
```

```

15  Adult_flag                                11760 non-null
object
16  Daily_Avg_mins_spend_on_traveling_page    11760 non-null
int64
dtypes: float64(4), int64(6), object(7)
memory usage: 1.5+ MB

```

Descriptive Summary

```
df.describe().T
```

	count	mean \
UserID	11760.0	1.005880e+06
Yearly_avg_view_on_travel_page	11760.0	2.803452e+02
total_likes_on_outstation_checkin_given	11760.0	2.816742e+04
yearly_avg_Outstation_checkins	11760.0	8.196259e+00
member_in_family	11760.0	2.921344e+00
Yearly_avg_comment_on_travel_page	11760.0	7.479371e+01
total_likes_on_outofstation_checkin_received	11760.0	6.531699e+03
week_since_last_outstation_checkin	11760.0	3.203571e+00
monthly_avg_comment_on_company_page	11760.0	2.866156e+01
Daily_Avg_mins_spend_on_traveling_page	11760.0	1.381743e+01

	std	min
UserID	3394.963917	1000001.0
Yearly_avg_view_on_travel_page	66.511330	35.0
total_likes_on_outstation_checkin_given	14150.080463	3570.0
yearly_avg_Outstation_checkins	8.650950	1.0
member_in_family	1.044883	1.0
Yearly_avg_comment_on_travel_page	23.815280	3.0
total_likes_on_outofstation_checkin_received	4706.613785	1009.0
week_since_last_outstation_checkin	2.616365	0.0
monthly_avg_comment_on_company_page	48.660504	11.0
Daily_Avg_mins_spend_on_traveling_page	9.070657	0.0

	25%	50% \
UserID	1002940.75	1005880.5
Yearly_avg_view_on_travel_page	233.00	271.0
total_likes_on_outstation_checkin_given	16697.25	28076.0

yearly_avg_Outstation_checkins	1.00	4.0
member_in_family	2.00	3.0
Yearly_avg_comment_on_travel_page	57.00	75.0
total_likes_on_outofstation_checkin_received	2940.75	4948.0
week_since_last_outstation_checkin	1.00	3.0
monthly_avg_comment_on_company_page	17.00	22.0
Daily_Avg_mins_spend_on_traveling_page	8.00	12.0

	75%	max
UserID	1008820.25	1011760.0
Yearly_avg_view_on_travel_page	322.00	464.0
total_likes_on_outstation_checkin_given	40115.25	252430.0
yearly_avg_Outstation_checkins	14.00	29.0
member_in_family	4.00	10.0
Yearly_avg_comment_on_travel_page	92.00	815.0
total_likes_on_outofstation_checkin_received	8393.25	20065.0
week_since_last_outstation_checkin	5.00	11.0
monthly_avg_comment_on_company_page	27.00	500.0
Daily_Avg_mins_spend_on_traveling_page	18.00	270.0

- 1. User Engagement Metrics: Users exhibit varied engagement levels on the platform, as reflected in metrics such as yearly average views on the travel page, total likes on outstation check-ins, and yearly average comments on the travel page.
- 2. User Demographics: User demographics indicate an average family size of approximately 2.88 members, with variability observed.
- 3. Check-in Behavior: Users tend to engage in an average of 8.10 outstation check-ins per year, with some variability in behavior. The duration since the last outstation check-in shows variability, potentially influencing current engagement levels.
- - a. Social Interaction: Social interaction plays a significant role, as seen in the metrics for total likes on out-of-station check-ins received and monthly comments on the company page. These metrics show the importance of user interactions within the platform.
- 5. Platform Usage Patterns: Users spend an average of 13.64 minutes daily on the traveling page, with variability in daily engagement.

```
df.select_dtypes(include = ['object']).describe().T
```

	count	unique	top	freq
Taken_product	11760	2	No	9864
preferred_device	11760	2	Mobile	10652
preferred_location_type	11760	15	Beach	2455
following_company_page	11760	4	0	8458
working_flag	11760	2	No	9952
travelling_network_rating	11760	4	3	3672
Adult_flag	11760	4	0	5048

-

- a. User Preferences and Behavior: The majority of users have not taken a specific product ("Taken_product: No") and prefer accessing the platform through mobile devices ("Preferred_device: Mobile"). This suggests the data is imbalanced.
-
- a. Location Preferences: The most frequent preferred location type is the "Beach", indicating a notable interest in beach-related destinations among users.
-
- a. Social Interaction and Employment Status: A significant portion of users is not actively following company pages ("Following_company_page: 0") and may currently be unemployed ("Working_flag: No").

Data Visualization

```
import plotly.express as px

# Distribution of Target Variable
fig = px.pie(
    df.assign(ClassMap=df.Taken_product.map({'No': "Not Taken", "Yes":
    "Taken"})),
    names="ClassMap", hole=0.5, color_discrete_sequence=["#79a5db",
    "#e0a580"])
fig.update_layout(height=450,width=600,
font_color="#28838a",title_font_size=16, showlegend=False,)
fig.add_annotation( x=0.5, y=0.5, align="center",
xref="paper",yref="paper", showarrow=False, font_size=20,
text="Target<br>Overview",)
fig.update_traces(hovertemplate=None, textposition="outside",
texttemplate="%{label}<br>%{value} - %{percent}",
textfont_size=16,rotation=-20, marker_line_width=25,
marker_line_color='#ffffff',)
fig.show()
```



Distribution of Numerical variables

- Let's check an overview of how each feature is distributed. We'll plot a histogram to check the distribution outliers. Further, we'll print the descriptive statistics of each feature along with Skewness & Kurtosis

```
import matplotlib.pyplot as plt
import seaborn as sns

import matplotlib.pyplot as plt
import seaborn as sns

# Define colours (using seaborn color palette)
colours = sns.color_palette("husl", len(num_columns)) # Ensures
enough colors for columns

plt.rcParams['axes.facecolor'] = 'white'
fig = plt.figure(figsize=[40, 20])
fig.suptitle('DISTRIBUTION OF DATA', fontsize=18, fontweight='bold')
fig.subplots_adjust(top=0.92, hspace=0.5, wspace=0.4)

for i, col in enumerate(num_columns):
    ax = fig.add_subplot(3, 3, i + 1)

    # Use sns.histplot instead of deprecated sns.distplot
    sns.histplot(df[col], color=colours[i], kde=True, ax=ax)

    # Vertical lines for quartiles, mean, and median
    ax.axvline(df[col].quantile(0.25), color='green', linestyle='--',
label='25% Quartile')
    ax.axvline(df[col].mean(), color='red', linestyle='--',
label='Mean')
    ax.axvline(df[col].median(), color='black', linestyle='--',
label='Median')
    ax.axvline(df[col].quantile(0.75), color='blue', linestyle='--',
label='75% Quartile')

    # Compute skewness & kurtosis
    skewness = round(df[col].skew(), 2)
    kurtosis = round(df[col].kurtosis(), 2)

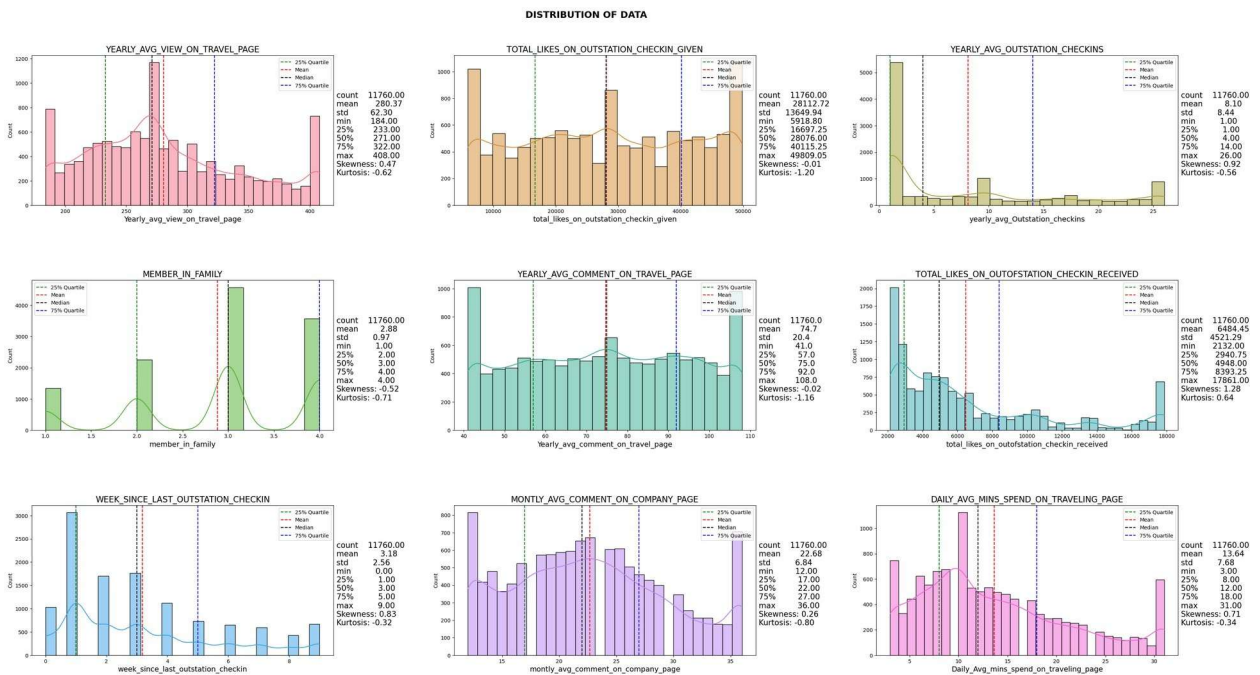
    # Generate summary text
    description_lines = [line for line in
str(df[col].describe().round(2)).split('\n') if 'Name' not in line and
'dtype' not in line]
    description_text = '\n'.join(description_lines) + f"\nSkewness:
{skewness:.2f}\nKurtosis: {kurtosis:.2f}"

    # Add annotation for summary stats
    ax.annotate(description_text, xy=(1.01, 0.2), xycoords='axes
fraction', fontsize=15)
```

```
# Set labels and title
ax.set_xlabel(col, fontsize=14)
ax.set_title(col.upper(), fontsize=16)

ax.legend(fontsize=10)

plt.show()
```



Distribution of Categorical variables

```
df.travelling_network_rating.value_counts()
```

```
travelling_network_rating
3      3672
4      3456
2      2424
1      2208
Name: count, dtype: int64
```

- In network travelling_network_rating, it shows 1 value having 10, let's drop it

```
df= df[df['travelling_network_rating'] != 10]
```

```
cat_columns = df.select_dtypes(include = ['object'])
```

```
def univariateAnalysis_category(cols):
    print("Distribution of", cols)
```

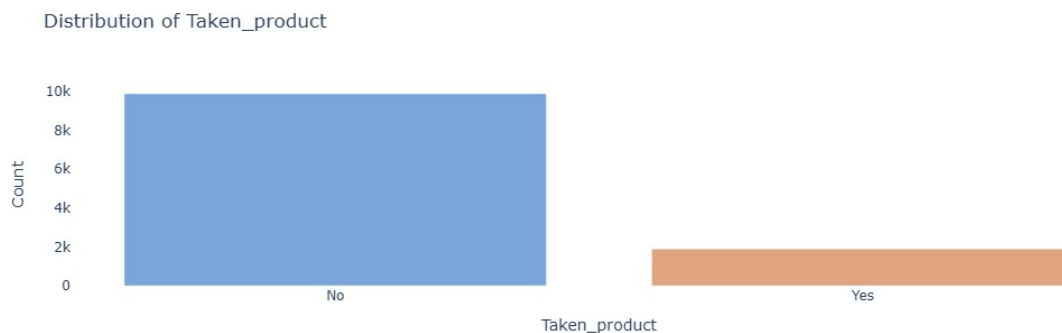
```
print("-----")
print("-")
```

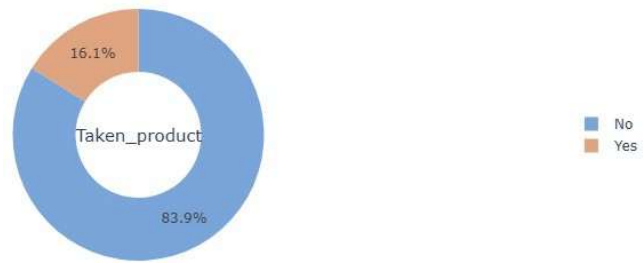
```

colors = ['#79a5db', '#e0a580', '#6fab90', '#896ca8', '#ADD8E6']
value_counts = cat_columns[cols].value_counts()
# Count plot
fig = px.bar(
    value_counts,
    x=value_counts.index,
    y=value_counts.values,
    title=f'Distribution of {cols}',
    labels={'x': 'Categories', 'y':
'Count'},color_discrete_sequence=[colors])
fig.update_layout(width=700)
fig.update_layout(plot_bgcolor='#ffffff', paper_bgcolor='#ffffff')
fig.show()
# Donut chart
percentage = (value_counts / value_counts.sum()) * 100
fig = px.pie(
    values=percentage, names=value_counts.index,
    labels={'names': 'Categories', 'values': 'Percentage'},
hole=0.5,color_discrete_sequence=colors)
fig.add_annotation(
    x=0.5, y=0.5, align="center", xref="paper",
    yref="paper", showarrow=False, font_size=15, text=f'{cols}')
fig.update_layout(legend=dict(x=0.9, y=0.5))
fig.update_layout(width=700)
fig.show()
print(" ")
for x in cat_columns:
    univariateAnalysis_category(x)

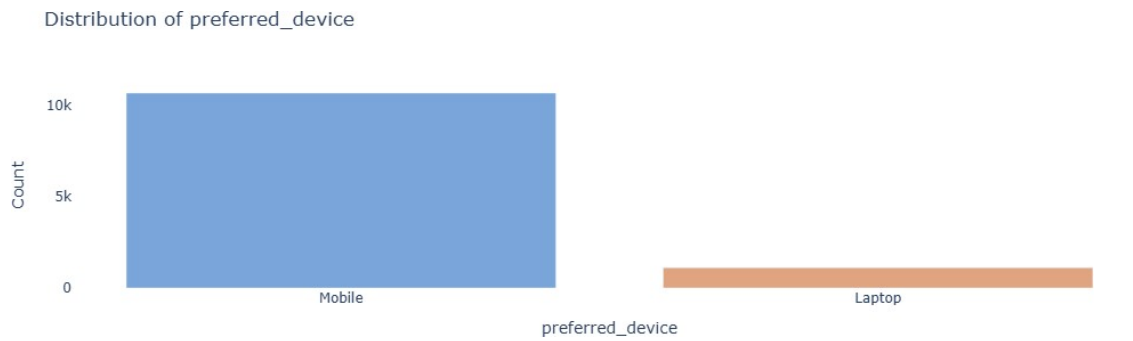
```

Distribution of Taken_product



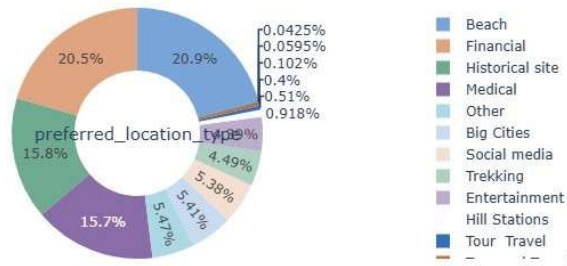


Distribution of preferred_device

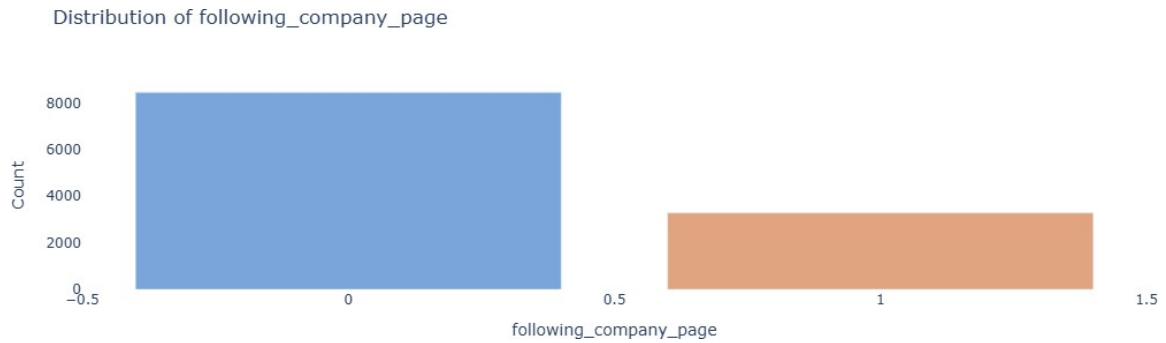


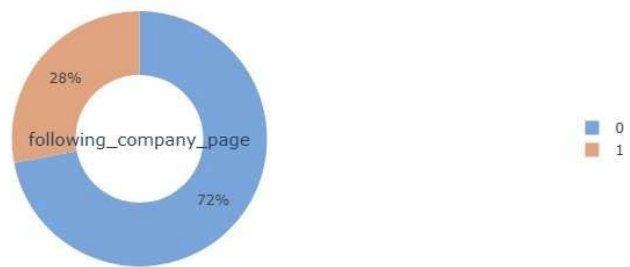
Distribution of preferred_location_type

Distribution of preferred_location_type

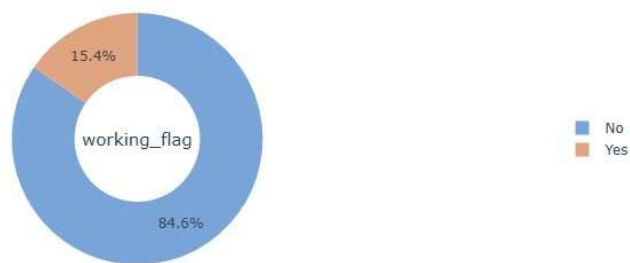
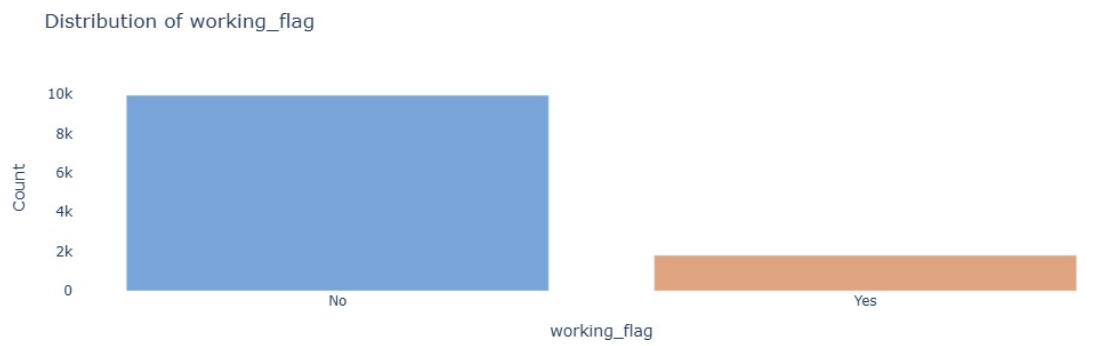


Distribution of following_company_page



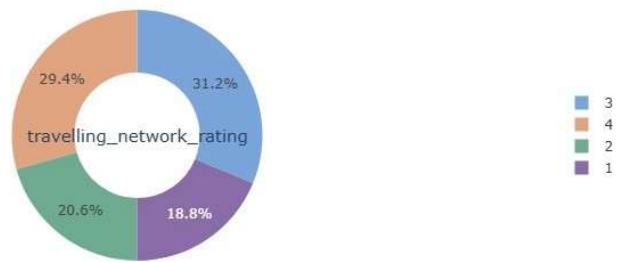
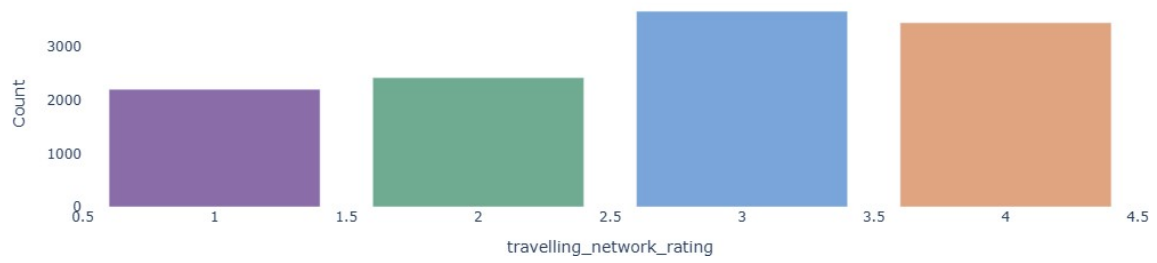


Distribution of working_flag

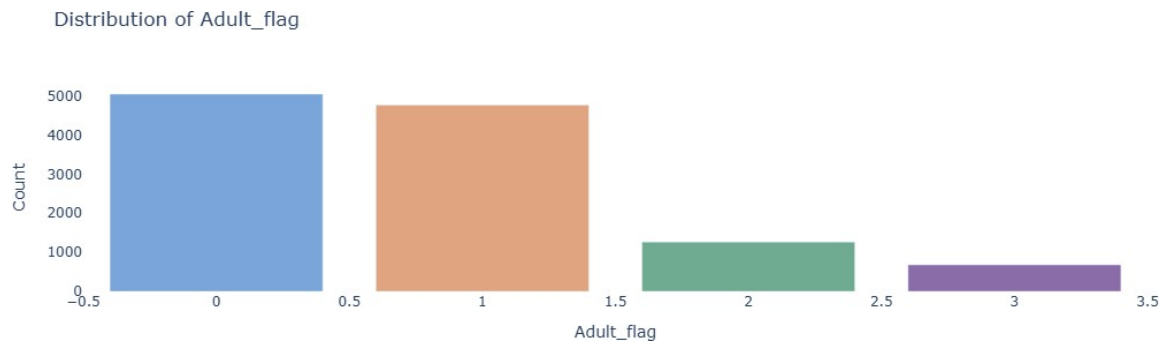


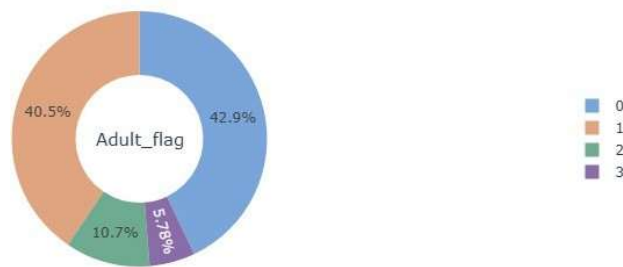
Distribution of travelling_network_rating

Distribution of travelling_network_rating



Distribution of Adult_flag





- 1.Digital Engagement: The high preference for Mobile usage (90.58%) suggests a strong emphasis on digital engagement.
- 2.Travel Preferences: Travel location preferences exhibit diversity, with top choices including Beach, Financial areas, and Historical sites.
- 3.User Interaction with Company: A significant proportion, 71.93%, does not follow the company page, emphasizing the need for strategic efforts to boost engagement and brand loyalty.
- 4.Occupational Influence: The majority of users, 84.62%, are not employed

Bivariate Analysis of Numeric Features

```
import matplotlib.pyplot as plt
import seaborn as sns

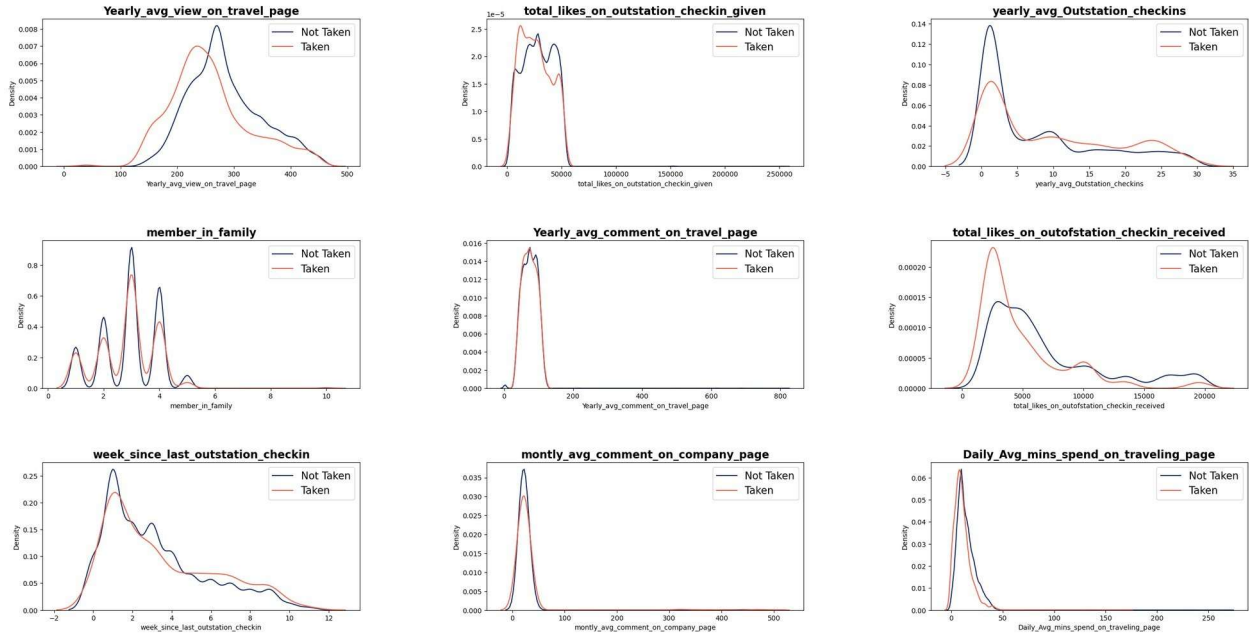
fig = plt.figure(figsize=[32, 15])
fig.suptitle('Bivariate Analysis : Distribution of Columns with
Product Taken', fontsize=18, fontweight='bold')
fig.subplots_adjust(top=0.92, hspace=0.5, wspace=0.4)

# Define title style dictionary
axtitl = {'fontsize': 16, 'fontweight': 'bold'}

for i, col in enumerate(num_columns, 1): # Use 1-based index
    a = fig.add_subplot(3, 3, i)
    sns.kdeplot(df[df['Taken_product'] == "No"][col], color='#142863',
ax=a, label='Not Taken')
    sns.kdeplot(df[df['Taken_product'] == "Yes"][col],
color='#f2634e', ax=a, label='Taken')
    a.set_title(col, fontdict=axtitl)
    a.legend(fontsize=15)

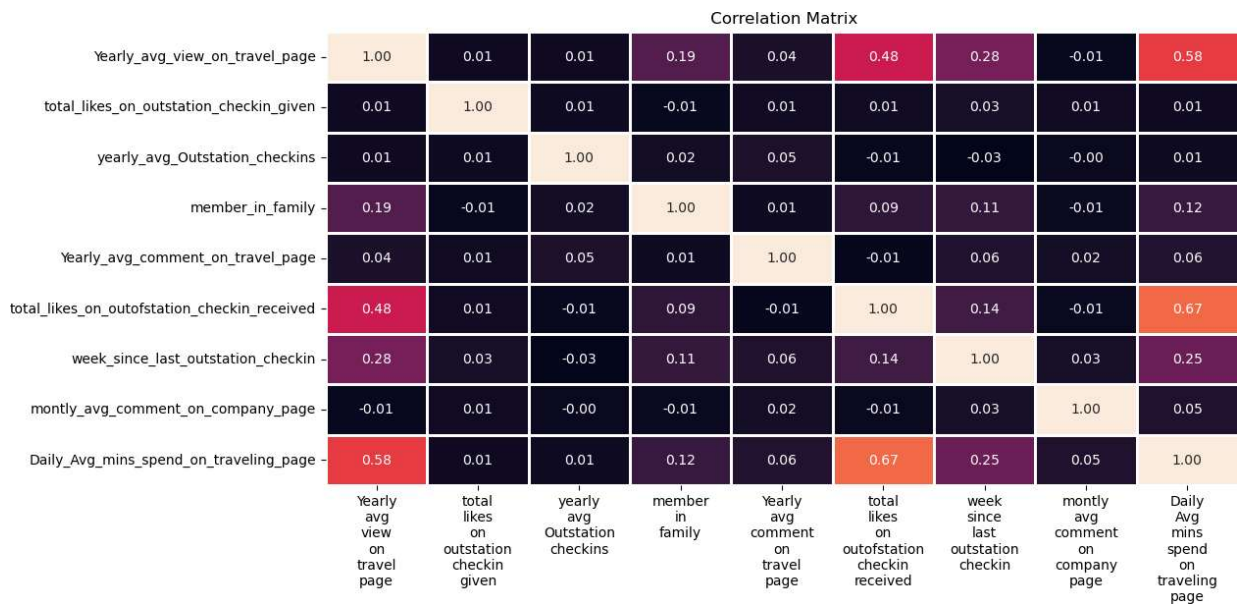
plt.show()
```


Bivariate Analysis : Distribution of Columns with Product Taken



Correlation heatmap

```
#Correlation heatmap
corr = df[num_columns].corr(method='pearson')
fig = plt.subplots(figsize=(12, 6))
ax = sns.heatmap(corr, annot=True, fmt='.2f', cbar=None,
linewidth=0.9)
ax.set_xticklabels([label.get_text().replace('_', '\n') for label in
ax.get_xticklabels()], rotation=0, horizontalalignment='center')
ax.set_title(' Correlation Matrix')
plt.show()
```



1. Engagement Patterns:

Users who spend more time viewing travel content on the platform also tend to spend more time engaging with travel-related activities, as evidenced by the moderate positive correlation between yearly average views on the travel page and daily average minutes spent on the traveling page (0.58).

2. Social Interaction Impact:

Social interaction plays a significant role in user engagement, with a strong positive correlation (0.67) observed between total likes on out-of-station check-ins received and the daily average minutes spent on the traveling page. This indicates that users receiving more likes on their check-ins are likely to spend more time on the platform.

3. Content Diversity and Family Dynamics:

There is a weak positive correlation (0.19) between the yearly average views on the travel page and the number of family members, suggesting a slight association between family dynamics and engagement with travel content. Additionally, the correlation with weeks since the last out-station check-in (0.28) indicates that user engagement might vary based on the recency of travel-related activities.

Outlier Treatment

```
import seaborn as sns
import matplotlib.pyplot as plt

axtitle_dict = {"fontsize": 14, "fontweight": "bold"}
axlab_dict = {"fontsize": 12}

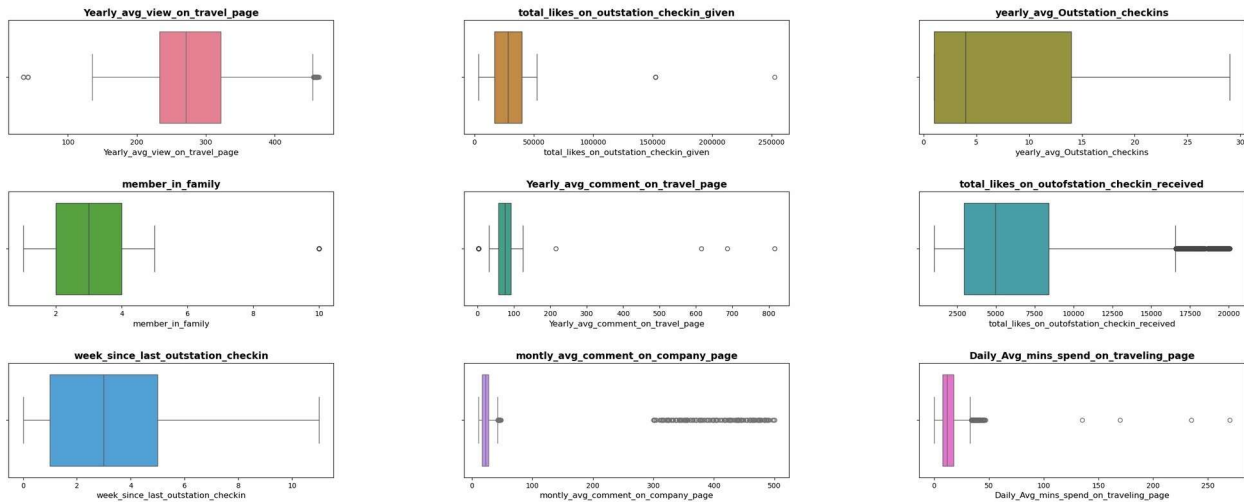
# Define colors (adjust as needed)
colours = sns.color_palette("husl", len(num_columns)) # Generates a
color for each column

plt.rcParams['axes.facecolor'] = 'white'
fig = plt.figure(figsize=[32, 24])
fig.suptitle('BOXPLOT OF ALL COLUMNS', fontsize=18, fontweight='bold')
fig.subplots_adjust(top=0.92, hspace=0.5, wspace=0.4)

for i, col in enumerate(num_columns):
    ax1 = fig.add_subplot(6, 3, i + 1)
    sns.boxplot(data=df, x=col, color=colours[i], ax=ax1) # Assign
color and set ax=ax1
    ax1.set_title(f'{col}', fontdict=axtitle_dict)
    ax1.set_xlabel(f'{col}', fontdict=axlab_dict)

plt.show()
```

BOXPLOT OF ALL COLUMNS



```
#Checking numbers of observations beyond Upper & Lower Limit
Q5 = df[num_columns].quantile(0.05)
Q95 = df[num_columns].quantile(0.95)
UL = Q95
LL = Q5
outliers = ((df[num_columns] > UL) | (df[num_columns] < LL)).sum()
print("Number of Observations Beyond Upper & Lower Limit for Each Column:")
display(outliers)
```

Number of Observations Beyond Upper & Lower Limit for Each Column:

Yearly_avg_view_on_travel_page	1168
total_likes_on_outstation_checkin_given	1176
yearly_avg_Outstation_checkins	491
member_in_family	395
Yearly_avg_comment_on_travel_page	1122
total_likes_on_outofstation_checkin_received	1170
week_since_last_outstation_checkin	198
montly_avg_comment_on_company_page	1006
Daily_Avg_mins_spend_on_traveling_page	1040

dtype: int64

, df[i])

```
#Function which returns the Upper and Lower limit to detect outliers
for each feature
def treat_outlier(col):
    q5 , q95 = np.percentile(col, [5, 95])
    return q5, q95

for i in num_columns:
```

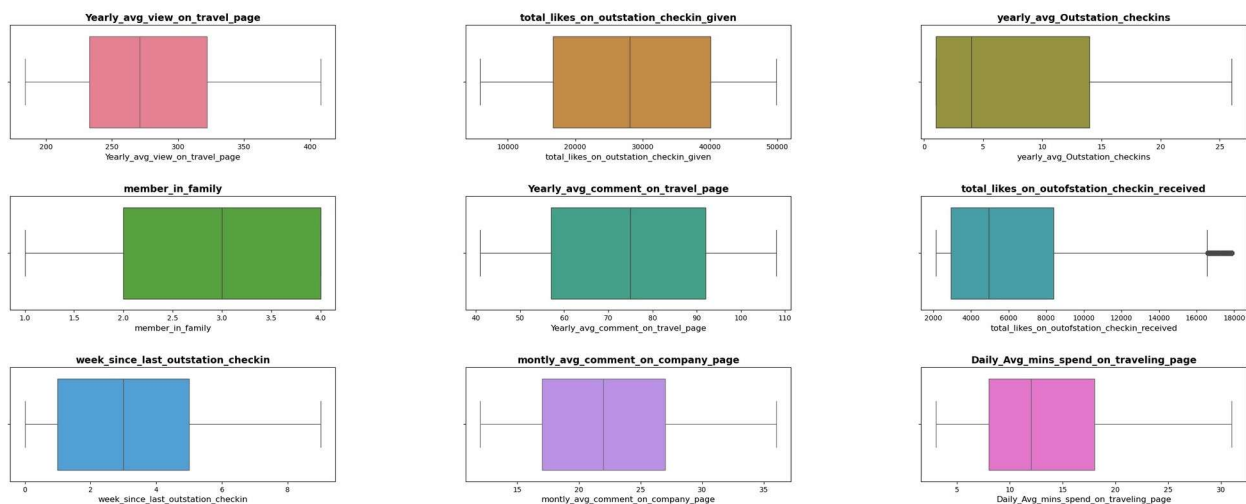
```

LR, UR = treat_outlier(df[i])
df[i] = np.where(df[i] > UR, UR, df[i])
df[i] = np.where(df[i] < LR, LR, df[i])

#Outliers in each Columns
plt.rcParams['axes.facecolor'] = 'white'
fig = plt.figure(figsize=[32,24])
fig.suptitle('BOXPLOT OF ALL COLUMNS POST TREATMENT (SCALED)',
fontsize=18, fontweight='bold')
fig.subplots_adjust(top=0.92);
fig.subplots_adjust(hspace=0.5, wspace=0.4);
for i ,col in enumerate(num_columns):
    ax1 = fig.add_subplot(6,3, i+1);
    ax1 = sns.boxplot(data = df, x=col , color= colours[i]);
    ax1.set_title(f'{col}', fontdict=axtitle_dict)
    ax1.set_xlabel(f'{col}', fontdict=axlab_dict)

```

BOXPLOT OF ALL COLUMNS POST TREATMENT (SCALED)



Inferences from EDA

- Device Preference : With 90.58% of users preferring mobile devices, prioritize mobile optimization for all digital content, including advertisements, website, and applications
- Enhancing Engagement: The majority of users (71.93%) do not follow the company page, indicating a potential gap in user engagement. Develop strategies to increase company page followers by providing valuable and relevant content. Engage users through interactive posts, surveys, and promotions to foster a sense of community and loyalty
- - a. Understanding Non-Working Majority: The dominance of non-working users (84.62%) suggests the need for tailoring marketing strategies. Consider offering flexible travel packages, discounts during off-peak hours, or special promotions for this user segment.
 - b. Addressing Travelling Network Ratings: The distribution of ratings, particularly with a significant percentage at '3' (31.23%) and '4' (29.38%), highlights the

importance of addressing user feedback. Implement improvements to enhance overall satisfaction and provide a positive user experience

- c. Demographic Tailoring for User Segments: Considering the diverse distribution in the Adult Flag category, with significant proportions in '0.0' (42.92%) and '1.0' (40.55%), tailoring services and promotions based on user age groups can lead to more effective and targeted marketing efforts.

The data is cleaned & ready to be used for Modeling post pre-processing

```
df= df.copy()

df.head()

  UserID Taken_product  Yearly_avg_view_on_travel_page
preferred_device \
0  1000001           Yes                               307.0
Mobile
1  1000002           No                               367.0
Mobile
2  1000003           Yes                               277.0
Mobile
3  1000004           No                               247.0
Mobile
4  1000005           No                               202.0
Mobile

  total_likes_on_outstation_checkin_given
yearly_avg_Outstation_checkins \
0                                     38570.0
1.0
1                                     9765.0
1.0
2                                     48055.0
1.0
3                                     48720.0
1.0
4                                     20685.0
1.0

  member_in_family preferred_location_type \
0           2.0           Financial
1           1.0           Financial
2           2.0           Other
3           4.0           Financial
4           1.0           Medical

  Yearly_avg_comment_on_travel_page \
0                               94.0
1                               61.0
```

2	92.0
3	56.0
4	41.0

	total_likes_on_outofstation_checkin_received \
0	5993.0
1	5130.0
2	2132.0
3	2909.0
4	3468.0

	week_since_last_outstation_checkin	following_company_page \
0	8.0	1
1	1.0	0
2	6.0	1
3	1.0	1
4	9.0	0

	monthly_avg_comment_on_company_page	working_flag
0	12.0	No
1	23.0	Yes
4	15.0	No
2	12.0	No
3	12.0	No
4	12.0	No

	Adult_flag	Daily_Avg_mins_spend_on_traveling_page
0	0	8.0
1	1	10.0
2	0	7.0
3	0	8.0
4	1	6.0

Step 2: Data Transformation

- Normalize numerical features (e.g., purchase_amount, ad_spend).
- Encode categorical variables using OneHotEncoder or LabelEncoder.
- Convert time-based data (e.g., ad_click_time, purchase_time) into useful formats (day, hour, etc.).

```
from sklearn.preprocessing import LabelEncoder, MinMaxScaler

# List of columns to encode
columns_to_encode = ['Taken_product', 'preferred_device',
'preferred_location_type', 'working_flag']
```

```

# Initialize LabelEncoder
label_encoder = LabelEncoder()

# Encode categorical columns
for column in columns_to_encode:
    df[column] = label_encoder.fit_transform(df[column])

# Display the encoded DataFrame
df.head(5)

```

	UserID	Taken_product	Yearly_avg_view_on_travel_page
preferred_device \			
0	1000001	1	307.0
1			
1	1000002	0	367.0
1			
2	1000003	1	277.0
1			
3	1000004	0	247.0
1			
4	1000005	0	202.0
1			

	total_likes_on_outstation_checkin_given
yearly_avg_Outstation_checkins \	
0	38570.0
1.0	
1	9765.0
1.0	
2	48055.0
1.0	
3	48720.0
1.0	
4	20685.0
1.0	

	member_in_family	preferred_location_type \
0	2.0	
1	1.0	
2	2.0	
3	4.0	
4	1.0	

	Yearly_avg_comment_on_travel_page \
0	94.0
1	61.0
2	92.0
3	56.0
4	41.0

```

total_likes_on_outofstation_checkin_received \
0          5993.0
1          5130.0
2          2132.0
3          2909.0
4          3468.0

week_since_last_outstation_checkin following_company_page \
0          8.0          1
1          1.0          0
2          6.0          1
3          1.0          1
4          9.0          0

monthly_avg_comment_on_company_page working_flag
travelling_network_rating \
0          12.0          0
1
1          23.0          1
4
2          15.0          0
2
3          12.0          0
3
4          12.0          0
4

Adult_flag  Daily_Avg_mins_spend_on_traveling_page
0          0          8.0
1          1          10.0
2          0          7.0
3          0          8.0
4          1          6.0

df.shape

(11760, 17)

# Separate features (X) and target variable (y)
features=df.drop('Taken_product', axis=1)
X = df.drop('Taken_product', axis=1)
y = df['Taken_product']

from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

```



```
!pip install xgboost
```

```
Requirement already satisfied: xgboost in c:\users\kajay\anaconda3\lib\site-packages (2.1.4)
```

```
Requirement already satisfied: numpy in c:\users\kajay\anaconda3\lib\site-packages (from xgboost) (1.26.4)
```

```
Requirement already satisfied: scipy in c:\users\kajay\anaconda3\lib\site-packages (from xgboost) (1.13.1)
```

Step 3: Model Development

Task 1: User Segmentation for Personalized Marketing

- Goal: Cluster users based on travel behavior.
- Approach:
- Use K-Means Clustering to segment users into groups.
- Identify clusters based on travel engagement, device preference, and travel check-ins.

```
from sklearn.cluster import KMeans

# Selecting relevant features for segmentation
features = df[['Yearly_avg_view_on_travel_page',
'yearly_avg_Outstation_checkins',
               'travelling_network_rating',
'Daily_Avg_mins_spend_on_traveling_page']]

# Applying K-Means Clustering
kmeans = KMeans(n_clusters=4, random_state=42)
df['customer_segment'] = kmeans.fit_predict(features)

df.head()
```

	UserID	Taken_product	Yearly_avg_view_on_travel_page
0	1000001	1	307.0
1			
1	1000002	0	367.0
1			
2	1000003	1	277.0
1			
3	1000004	0	247.0
1			
4	1000005	0	202.0
1			

```
total_likes_on_outstation_checkin_given
```

yearly_avg_Outstation_checkins	\
0	38570.0
1.0	
1	9765.0
1.0	
2	48055.0
1.0	
3	48720.0
1.0	
4	20685.0
1.0	

member_in_family	preferred_location_type	\
0	2.0	3
1	1.0	3
2	2.0	10
3	4.0	3
4	1.0	7

Yearly_avg_comment_on_travel_page	\
0	94.0
1	61.0
2	92.0
3	56.0
4	41.0

total_likes_on_outofstation_checkin_received	\
0	5993.0
1	5130.0
2	2132.0
3	2909.0
4	3468.0

2	2	0
3	3	0
4	4	1

```

Daily_Avg_mins_spend_on_traveling_page  customer_segment
0                                         8.0             3
1                                         10.0            1
2                                         7.0             2
3                                         8.0             2
4                                         6.0             0

# Visualizing clusters
plt.figure(figsize=(8, 6))
sns.scatterplot(x=df['Yearly_avg_view_on_travel_page'],
                y=df['travelling_network_rating'],
                hue=df['customer_segment'], palette='viridis')
plt.title("Customer Segments Based on Travel Preferences")
plt.show()

```



Task 2: Predicting Marketing ROI & Conversion Likelihood

- Goal: Use classification models to predict which customers will respond positively to marketing campaigns.
- Approach:
- Define the target variable (e.g., Taken_product as whether the user buys a ticket).
- Train a Random Forest Classifier to predict marketing response.

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# Define target and features
X = df.drop(columns=['UserID', 'Taken_product'])
y = df['Taken_product']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Train model
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

# Evaluate model
y_pred = rf_model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test,
y_pred))
```

Accuracy: 0.9906462585034014

Classification Report:

	precision	recall	f1-score	support
0	0.99	1.00	0.99	1958
1	1.00	0.94	0.97	394
accuracy			0.99	2352
macro avg	0.99	0.97	0.98	2352
weighted avg	0.99	0.99	0.99	2352

Task 3: Enhancing Digital Ad Targeting for Ticket Purchases

- Goal: Build separate models for laptop and mobile users to improve ad targeting.
- Approach:

- Filter dataset based on device type.
- Train separate XGBoost models for each group.

Test Model to predict

X_test

	Yearly_avg_view_on_travel_page	preferred_device	\
11054	223.0	1	
1825	195.0	1	
2087	330.0	1	
7920	394.0	1	
4681	354.0	1	
...	...	...	
1330	322.0	1	
10030	256.0	1	
957	300.0	1	
3334	234.0	1	
10196	295.0	1	

	total_likes_on_outstation_checkin_given	\
11054	40087.0	
1825	32655.0	
2087	21875.0	
7920	25032.0	
4681	23356.0	
...	...	
1330	28805.0	
10030	9400.0	
957	16030.0	
3334	16836.0	
10196	42257.0	

	yearly_avg_Outstation_checkins	member_in_family	\
11054	10.0	1.0	
1825	1.0	3.0	
2087	1.0	3.0	
7920	19.0	4.0	
4681	26.0	4.0	
...	...	...	
1330	1.0	3.0	
10030	21.0	3.0	
957	16.0	2.0	
3334	1.0	2.0	
10196	9.0	2.0	

	preferred_location_type	Yearly_avg_comment_on_travel_page	\
11054	6	91.0	
1825	3	75.0	
2087	0	50.0	

7920	6	50.0
4681	3	75.0
...	...	...
1330	7	81.0
10030	6	98.0
957	3	74.0
3334	7	45.0
10196	0	91.0

total_likes_on_outofstation_checkin_received \	
11054	2132.0
1825	5296.0
2087	5933.0
7920	4269.0
4681	11904.0
...	...
1330	17861.0
10030	2132.0
957	3544.0
3334	4353.0
10196	4831.0

week_since_last_outstation_checkin following_company_page \		
11054	3.0	0
1825	1.0	0
2087	9.0	0
7920	0.0	0
4681	3.0	1
...	...	...
1330	7.0	0
10030	1.0	0
957	9.0	0
3334	3.0	0
10196	4.0	0

montly_avg_comment_on_company_page working_flag \		
11054	36.0	1
1825	17.0	0
2087	12.0	0
7920	29.0	0
4681	20.0	0
...	...	...
1330	13.0	0
10030	33.0	0
957	16.0	0
3334	16.0	0
10196	36.0	0

travelling_network_rating Adult_flag \		
11054	2	0

1825	2	1
2087	4	0
7920	1	1
4681	4	1
...	...	...
1330	4	0
10030	4	3
957	2	1
3334	2	1
10196	4	2

	Daily_Avg_mins_spend_on_traveling_page	customer_segment
11054	8.0	0
1825	8.0	0
2087	10.0	3
7920	5.0	1
4681	14.0	3
...	...	...
1330	21.0	3
10030	16.0	2
957	6.0	3
3334	21.0	0
10196	10.0	3

[2352 rows x 16 columns]

```
x= X_test.loc[[11054]]
x
```

	Yearly_avg_view_on_travel_page	preferred_device \
11054	223.0	1

	total_likes_on_outstation_checkin_given \
11054	40087.0

	yearly_avg_Outstation_checkins	member_in_family \
11054	10.0	1.0

	preferred_location_type	Yearly_avg_comment_on_travel_page \
11054	6	91.0

	total_likes_on_outofstation_checkin_received \
11054	2132.0

	week_since_last_outstation_checkin	following_company_page \
11054	3.0	0

	montly_avg_comment_on_company_page	working_flag \
11054	36.0	1

	travelling_network_rating	Adult_flag \
--	---------------------------	--------------

```

11054                2                0

      Daily_Avg_mins_spend_on_traveling_page customer_segment
11054                                8.0                        0

rf_model.predict(x)

array([0], dtype=int64)

df.head()

   UserID Taken_product  Yearly_avg_view_on_travel_page
preferred_device \
0  1000001            1                               307.0
1
1  1000002            0                               367.0
1
2  1000003            1                               277.0
1
3  1000004            0                               247.0
1
4  1000005            0                               202.0
1

      total_likes_on_outstation_checkin_given
yearly_avg_Outstation_checkins \
0                                38570.0
1.0
1                                9765.0
1.0
2                                48055.0
1.0
3                                48720.0
1.0
4                                20685.0
1.0

   member_in_family  preferred_location_type \
0                2.0
1                1.0
2                2.0
3                4.0
4                1.0

   Yearly_avg_comment_on_travel_page \
0                                94.0
1                                61.0
2                                92.0
3                                56.0

```



```

4                                     41.0

total_likes_on_outofstation_checkin_received \
0                                     5993.0
1                                     5130.0
2                                     2132.0
3                                     2909.0
4                                     3468.0

week_since_last_outstation_checkin following_company_page \
0                                     8.0                                     1
1                                     1.0                                     0
2                                     6.0                                     1
3                                     1.0                                     1
4                                     9.0                                     0

monthly_avg_comment_on_company_page working_flag
travelling_network_rating \
0                                     12.0                                     0
1
1                                     23.0                                     1
4
2                                     15.0                                     0
2
3                                     12.0                                     0
3
4                                     12.0                                     0
4

Adult_flag  Daily_Avg_mins_spend_on_traveling_page  customer_segment
0           0                                     8.0                                     3
1           1                                     10.0                                    1
2           0                                     7.0                                     2
3           0                                     8.0                                     2
4           1                                     6.0                                     0

df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11760 entries, 0 to 11759
Data columns (total 18 columns):
 #   Column                                     Non-Null Count
Dtype  -----
-----

```

0	UserID	11760 non-null
int64		
1	Taken product	11760 non-null
int64		
2	Yearly_avg_view_on_travel_page	11760 non-null
float64		
3	preferred device	11760 non-null
int64		
4	total_likes_on_outstation_checkin_given	11760 non-null
float64		
5	yearly_avg_Outstation_checkins	11760 non-null
float64		
6	member in family	11760 non-null
float64		
	preferred location type	11760 non-null
int64		
8	Yearly_avg_comment_on_travel_page	11760 non-null
float64		
9	total_likes_on_outofstation_checkin_received	11760 non-null
float64		
10	week_since_last_outstation_checkin	11760 non-null
float64		
11	following company page	11760 non-null
object		
12	montly_avg_comment_on_company_page	11760 non-null
float64		
13	working flag	11760 non-null
int32		
	ravelling network rating	11760 non-null
object		
	Adult flag	11760 non-null
object		
16	Daily_Avg_mins_spend_on_traveling_page	11760 non-null
float64		
17	customer segment	11760 non-null
int32		

dtypes: float64(9), int32(2), int64(4), object(3)
memory usage: 1.5+ MB

```
df['following_company_page']= df['following_company_page'].astype(int)
df['travelling_network_rating']=
df['travelling_network_rating'].astype(int)
df['Adult_flag']= df['Adult_flag'].astype(int)
```

xgboost Model

```
import xgboost as xgb

# Split data for laptop and mobile users
```

```

df_laptop = df[df['preferred_device'] == 1]
df_mobile = df[df['preferred_device'] == 0]

# Prepare training data
def train_xgb_model(df):
    X = df.drop(columns=['UserID', 'Taken_product'])
    y = df['Taken_product']
    X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

    model = xgb.XGBClassifier(n_estimators=100, learning_rate=0.1,
random_state=42)
    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print("Classification Report:\n", classification_report(y_test,
y_pred))

# Train models for each device type
print("Laptop Users Model:")
train_xgb_model(df_laptop)

print("\nMobile Users Model:")
train_xgb_model(df_mobile)

```

Laptop Users Model:

Accuracy: 0.975598310652276

Classification Report:

	precision	recall	f1-score	support
0	0.97	1.00	0.99	1836
1	0.98	0.84	0.90	295
accuracy			0.98	2131
macro avg	0.98	0.92	0.95	2131
weighted avg	0.98	0.98	0.97	2131

Mobile Users Model:

Accuracy: 0.975598310652276

Classification Report:

	precision	recall	f1-score	support
0	0.97	1.00	0.99	1836
1	0.98	0.84	0.90	295
accuracy			0.98	2131
macro avg	0.98	0.92	0.95	2131
weighted avg	0.98	0.98	0.97	2131

Expected Outcome

- User Segmentation: Airlines can personalize marketing based on user clusters.
- ROI Prediction: Improved decision-making on budget allocation.
- Ad Targeting: Higher conversion rates with device-specific strategies.